SOLUTION AND WORKFLOW REPORT

Salesforce QA Agent

A Chrome extension and local AI agent that runs Salesforce test cases as real users, collects evidence, files defects and checks test cases against the requirements

VERSION
Extension 0.2.0 · Agent server 0.1.0

DATE
5 October 2026

PREPARED BY
Mayank Jain

ORG IN USE
DRIVE-ENERGY-4828 - TMCDEV (sandbox)

26	12	7	3
test runs recorded (1–5 Oct 2026)	distinct test cases executed	defects filed with screenshot evidence	management reports: execution, defects, test case review

Contents

1. Executive summary
2. System overview and architecture
3. Components
4. One-time setup workflow
5. Running a single test case
6. How a result is decided
7. Running a batch from Excel / CSV
8. Checking test cases against the BRD
9. Reports
10. Security and safeguards
11. Operating the agent
12. Limitations and next steps

1 Executive summary

The Salesforce QA Agent turns written test cases into executed, evidenced test results without a tester clicking through Salesforce. A tester opens any Salesforce page, picks the org and the persona (the Salesforce user the test should run as), pastes or uploads test cases, and the agent does the rest.

What it does

Runs tests as real users

Each test logs in as the persona's actual Salesforce user through the admin's **Login As**, in a separate, isolated browser. Nobody types or stores persona passwords.

AI executes the steps

Claude (through Claude Code) reads the test case, performs each step in the browser, verifies the result and records a PASS / FAIL / BLOCKED status for every step.

Evidence for every step

Every step has its own screenshot. A step without a screenshot is rejected, so results can always be audited.

Defects filed automatically

Failures become bug reports with steps to reproduce, expected vs actual, severity and screenshots. Repeat failures are linked to the existing bug instead of being filed twice.

Batches from Excel / CSV

A whole test sheet can be uploaded, checked row by row, and run one test after another, with a batch summary.

Test cases checked against the BRD

Before testing, the BRD or requirement sheet can be uploaded to check that each test case is correct and traceable, and to find requirements no test case covers.

Why it matters

- **Less manual effort:** testers write or upload test cases; execution, screenshots and defect write-ups are produced by the agent.
- **Consistent, auditable results:** the same rules decide every verdict, and every step carries evidence.
- **Earlier quality checks:** wrong or missing test cases are caught against the requirements before a test cycle starts.
- **Management visibility:** three plain-language reports show pass rate, open defects by severity and requirement coverage, and print to PDF.

> **Current status (5 October 2026).** In use on the DRIVE-ENERGY-4828 - TMCDEV sandbox. 26 runs of 12 test cases have been recorded, with 7 open defects (2 High, 5 Medium), most of them Sales-profile access issues on Account. All 127 automated tests pass, and an end-to-end check of the requirements review ran successfully against the real Claude CLI.

2 System overview and architecture

The solution has three parts that run on the tester's own Windows machine, plus the Salesforce org being tested. The extension is only the front end; all the work happens in the local agent server, which never accepts connections from the network (it listens on `127.0.0.1` only).

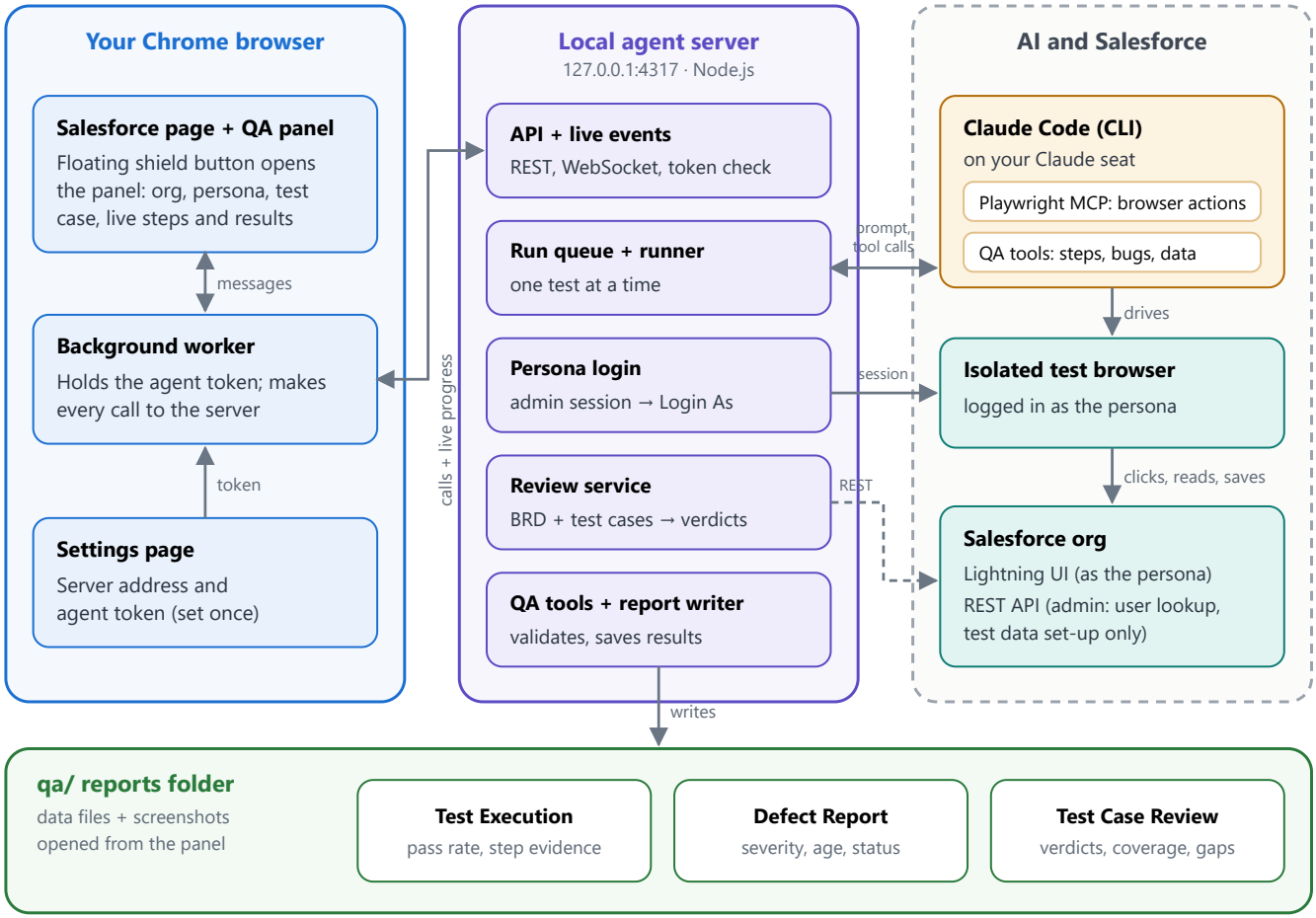


Figure 1. Architecture. The extension talks only to the local server. The server starts Claude Code with two tool servers: Playwright MCP to operate an isolated browser, and the QA tools through which Claude reports results back to the server. Results are saved in the qa/ folder and shown in three reports.

Chrome extension · Local agent server · Claude (AI) · Salesforce · Reports

These colours are used for the same parts in every diagram in this report.

Technology

TypeScript throughout, run directly by Node.js 22+; Chrome Manifest V3 extension built with esbuild; Fastify for the server; Playwright for browsers; Claude Code CLI with the Model Context Protocol (MCP) for the AI tools; Zod for validation; read-excel-file, fflate and unpdf to read Excel, Word and PDF files.

Where the data goes

Test cases, screenshots and results stay on the tester's machine in the project folder. The only external services are the Salesforce org under test and Claude (through the tester's Claude Code sign-in), which receives the test case or BRD text and page snapshots needed to do the work.

3 Components

COMPONENT	RESPONSIBILITY	WHERE
QA panel (content script)	Floating button and panel injected on Salesforce pages. Detects the org from the page address, manages personas, takes a pasted test case or an uploaded file, shows live steps, bugs and results, and links to the reports.	`packages/extension/src/content/`
Background worker	Holds the agent token and server address and makes every server call, so the Salesforce page never sees the token. Forwards the live event stream of a run to the panel.	`extension/src/background.ts`
Settings page	Where the tester pastes the agent token once and tests the connection.	`extension/src/options.ts`
Agent server	REST and WebSocket API on 127.0.0.1:4317. Registers orgs, saves admin sessions, searches users for personas, queues and runs tests, parses uploaded sheets, runs requirement reviews and serves the reports behind a secret link.	`packages/server/src/`
Runner	Executes one run at a time: persona login, hand-over to Claude, step and bug collection, verdict, report update, and clean-up of the persona session file.	`server/src/runs/runner.ts`
Persona login	Uses the org admin's saved session and Salesforce's Login As to open an isolated browser as the persona's user, and confirms the "Logged in as" banner.	`server/src/salesforce/`
Claude Code + MCP tools	Claude runs headless on the signed-in Claude Code seat (no API key billing). It only has two tool sets: Playwright MCP (browser) and the QA tools (record_step, report_bug, finish_test, test-data tools); for reviews, only the review tools.	`server/src/agent/`
Batch parser	Reads .xlsx / .csv test sheets, finds the header row, matches columns and personas loosely and flags invalid rows.	`server/src/suites/`
Review service	Extracts text from the BRD (PDF, Word, Excel, CSV, TXT, MD), runs Claude with the review tools, validates every verdict and saves the review.	`server/src/reviews/`
Reports	Three static report pages that read data files in the qa/ folder; status changes made by reviewers can be saved back into the data files.	`qa/reports, qa/bugs, qa/reviews`
Shared types	Schemas used by both extension and server, so requests are validated identically on both sides.	`packages/shared/`

4 One-time setup workflow

Setup is done once per machine (steps 1–2), once per org (step 4) and once per persona (step 5). After that, a tester only opens Salesforce and runs tests.

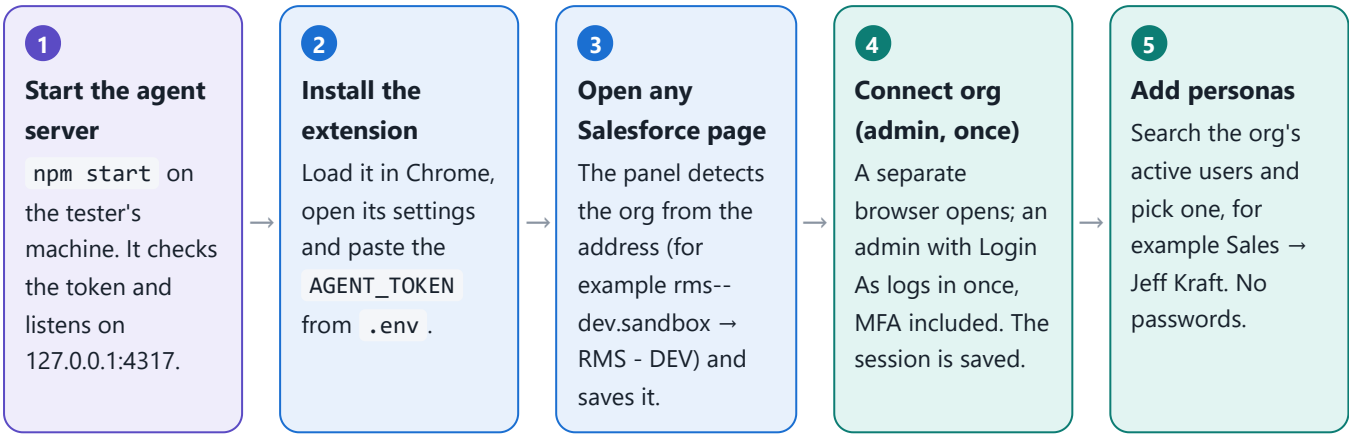


Figure 2. Setup steps. When Salesforce eventually expires the admin session, the panel shows **Reconnect**; only step 4 is repeated.

Why personas need no passwords

Personas are real Salesforce users. The agent logs in as them the same way an administrator clicks **Login** on a user record in Setup: it opens Salesforce's Login As address with the admin's saved session, in a brand-new browser that shares nothing with the tester's own Chrome.

Before opening any browser it checks, through the Salesforce API, that the admin session still works and that the persona's user exists and is active, so problems are reported in seconds with a clear fix.

What the panel shows

- Connection to the agent and the detected org
- Admin connection status, with **Connect** / **Reconnect**
- Persona picker; **+ Add** searches the org's users
- **Single test** tab: paste and run, with live steps, screenshots, bugs and log
- **Upload file** tab: batches and the requirements check

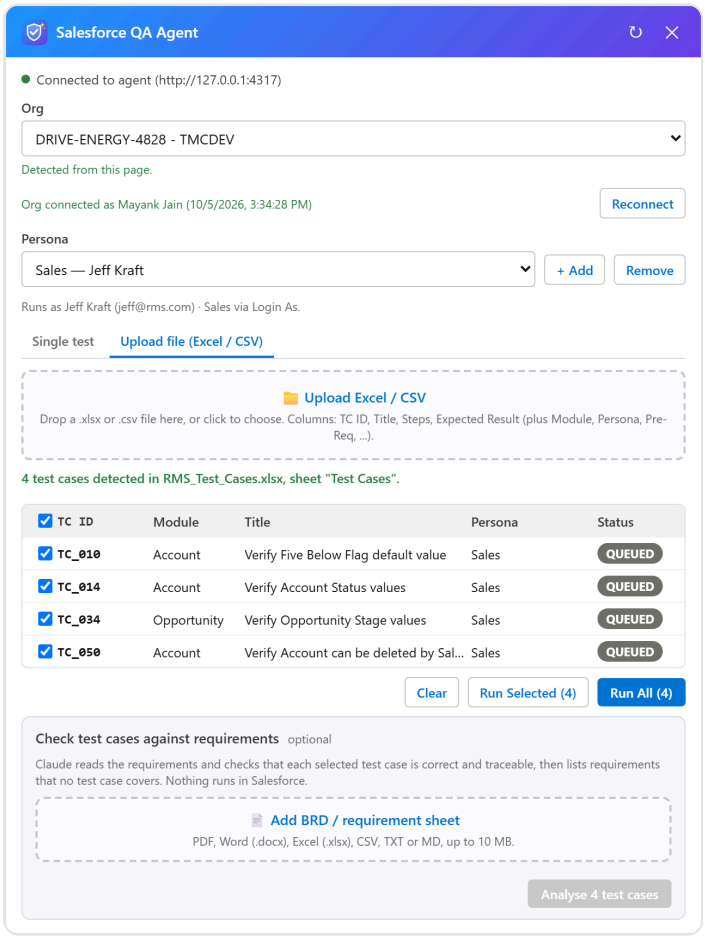


Figure 3. The QA panel (illustrative data) with an uploaded test file and the optional requirements check.

5 Running a single test case

The tester pastes a test case (it can be a row copied from a test sheet), optionally a title, and clicks **Run test**. The sequence below is what happens next. The panel updates live throughout, and the tester can keep working.

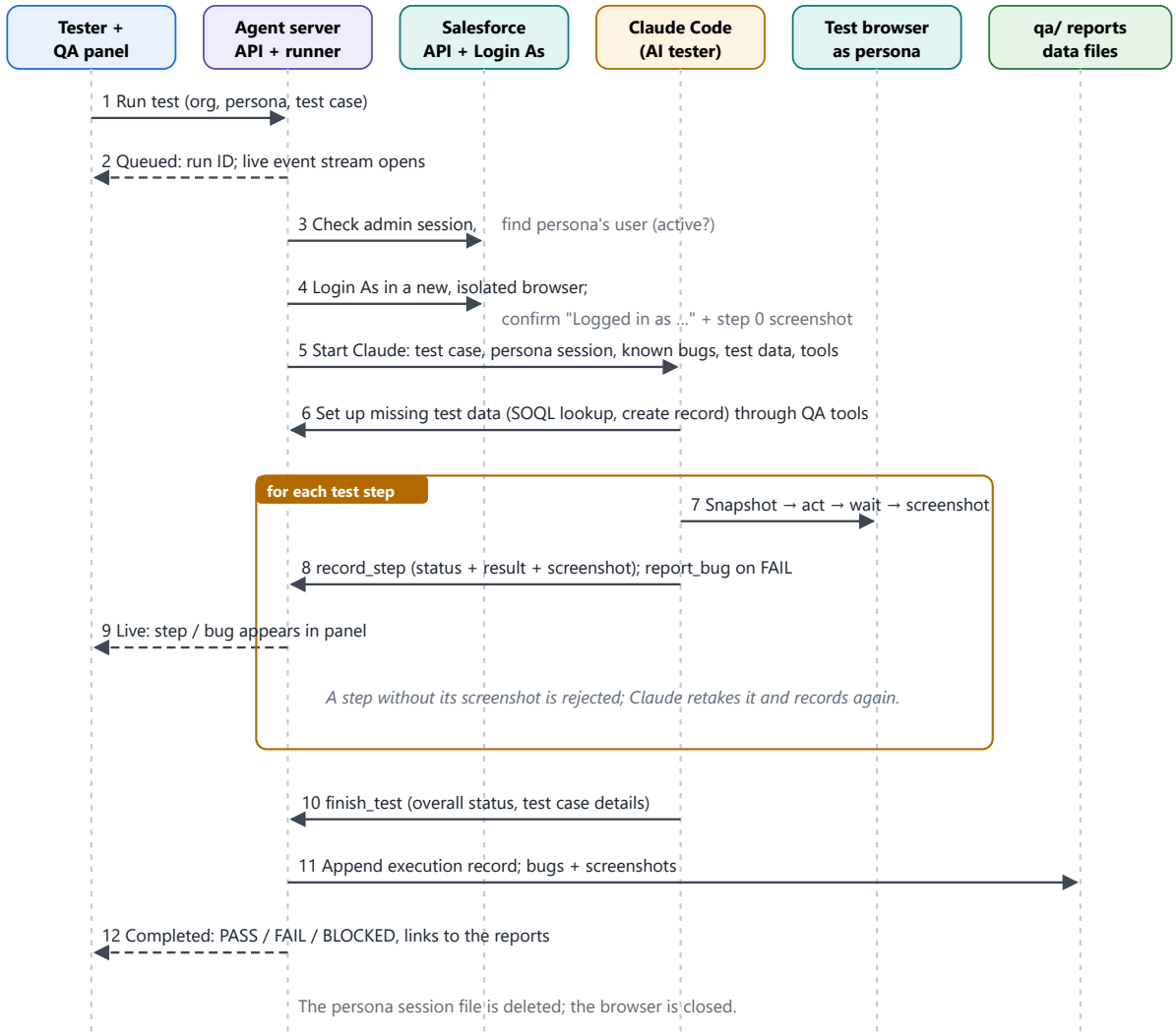


Figure 4. Sequence of one test run. Solid arrows are requests or tool calls; dashed arrows are live updates to the panel. Steps 7–8 repeat for every step of the test case.

What Claude is told to do

- Use only the browser tools and the QA tools. Never log out, switch user, change Setup configuration or delete records.
- Read the page before acting, use visible labels, and wait for Lightning spinners and toasts before judging a result.
- Verify thoroughly: list every picklist value seen, confirm saves after refresh, quote error messages exactly.
- Reuse test data created by earlier runs; create only what is missing, with the admin's API access, and only to prepare preconditions (never to perform the action being tested).
- Link a failure to a known open bug instead of filing a duplicate.

6 How a result is decided

Each step gets one status from Claude, defined in its instructions: **PASS** (behaves as expected), **FAIL** (the application does not behave as the test case expects, including a field that is hidden or read-only for the persona; a bug is filed), **BLOCKED** (cannot be tested for reasons outside the behaviour under test, such as test data that could not be created), or **N/A**. The overall verdict is then decided by fixed rules on the server, the same for the panel and the reports:

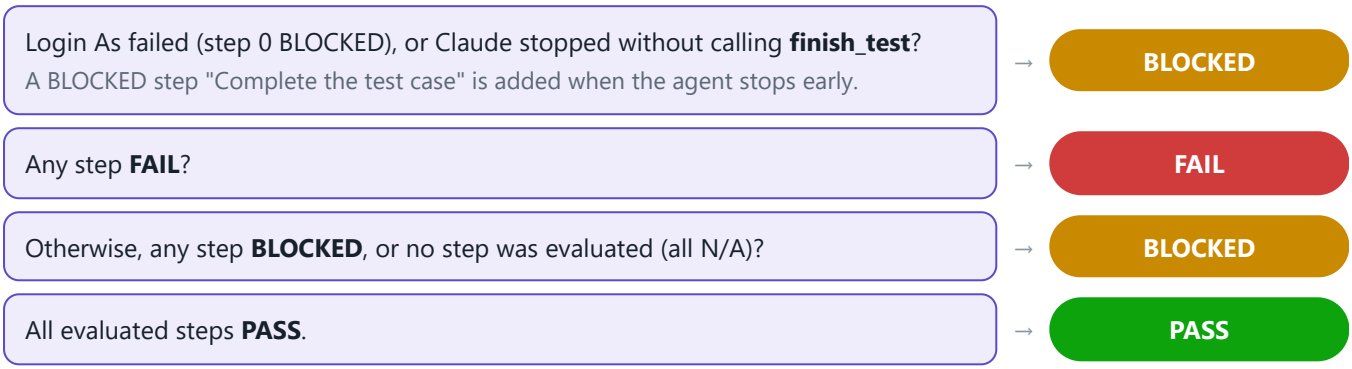


Figure 5. Verdict rules, checked top to bottom. Claude's own overall status can only make the verdict stricter (for example PASS → FAIL), never more lenient.

Evidence and defect rules

RULE	EFFECT
Every non-N/A step needs a screenshot	The server rejects `record_step` until the named screenshot exists, so every result can be checked.
A bug needs at least one screenshot	Bugs are filed with steps to reproduce, expected vs actual, severity (Critical / High / Medium / Low), priority and the record used; evidence is copied to `qa/bugs/<BUG-ID>/` .
Repeat failures	A failure matching a known open bug is recorded as a new occurrence of that bug (with its run and screenshots) rather than a duplicate.
Possible intentional behaviour	Filed with type "Requirement gap" (for example field-level security), so a reviewer can mark it "Not a Bug".
finish_test only once, only after steps	Calls in the wrong order are rejected with an explanation Claude can act on.

7 Running a batch from Excel / CSV

In the panel's **Upload file (Excel / CSV)** tab, a whole test sheet (up to 5 MB, 200 rows per batch) is turned into individual runs. Nothing runs until the tester has reviewed what was detected.

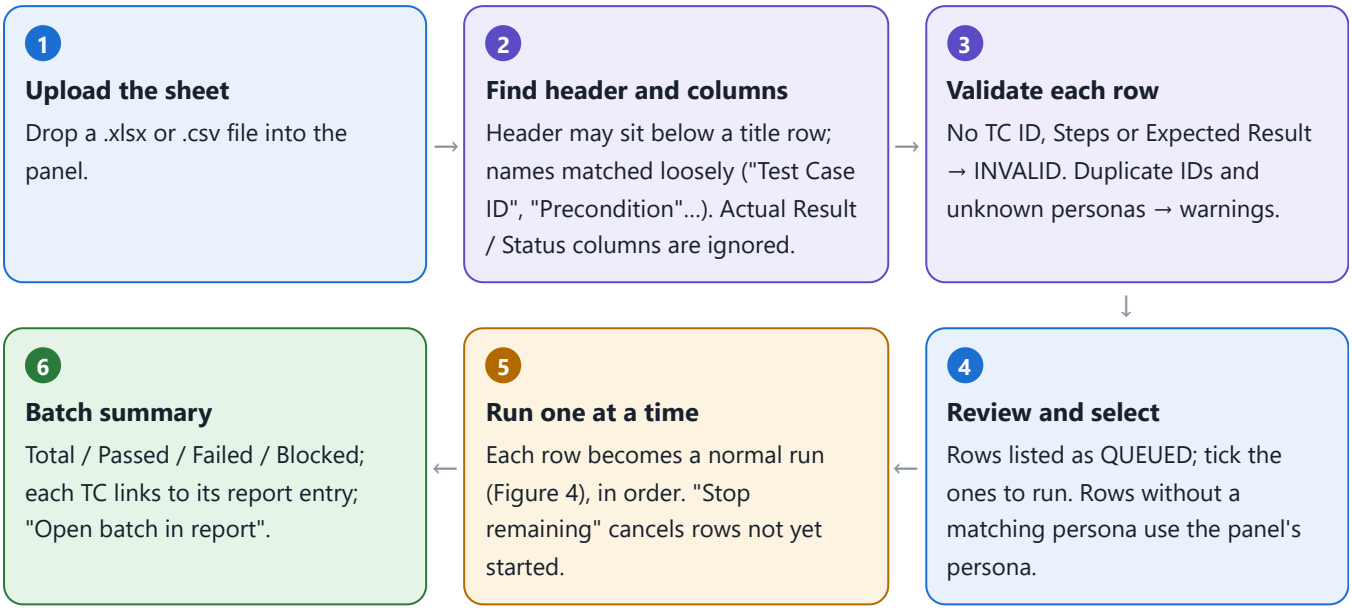


Figure 6. Batch workflow. If a required column is missing, the panel names it and nothing runs. Optional step: before step 5, the same selected rows can be checked against the BRD (section 8).

COLUMNS	ACCEPTED NAMES (EXAMPLES)
Required	TC ID, Title, Steps, Expected Result
Optional	Sprint, Story, Module, Persona, Description, Pre-Req / Precondition, Type, Priority, Test Data
Ignored	Actual Result, Status, Remarks, Executed By / On (results of an earlier execution are not instructions)

Runs execute strictly one after another, because every run shares the org admin's saved session; parallel browsers would compete for it.

8 Checking test cases against the BRD

After a test sheet is loaded, the optional **Check test cases against requirements** section accepts the BRD or requirement sheet. Claude reads both and judges every selected test case. Nothing runs in Salesforce and no org connection is needed; a review of a few test cases takes under a minute.

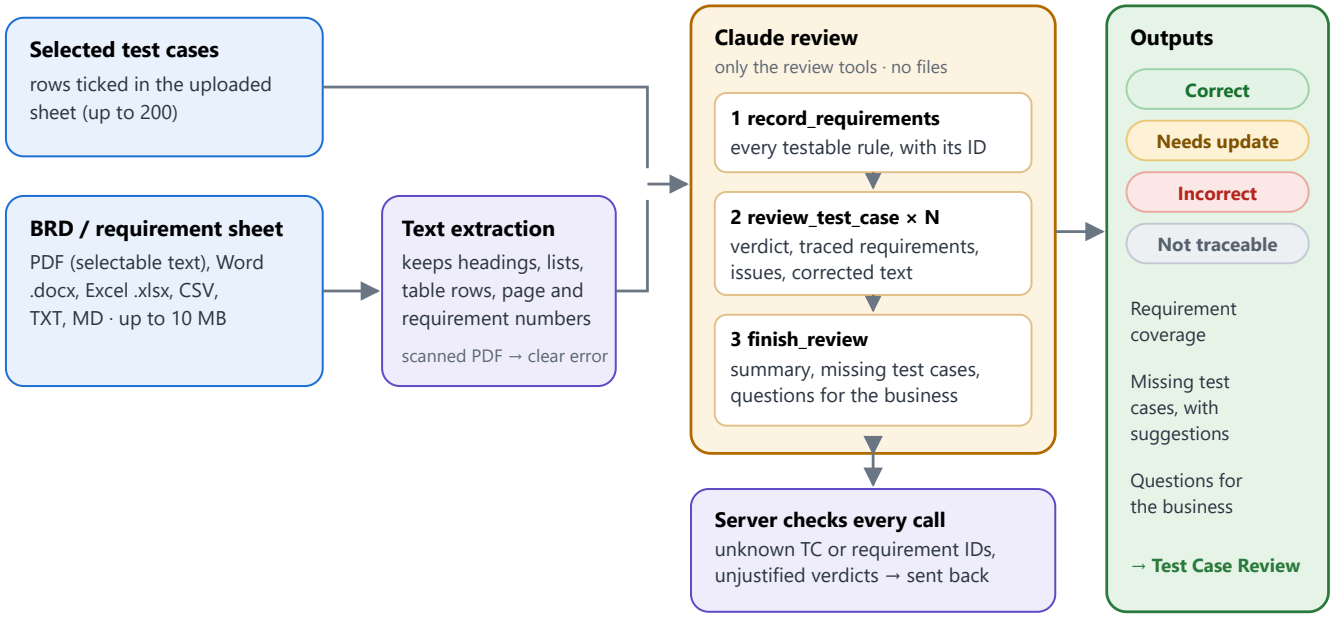


Figure 7. Requirements review. Progress shows live in the panel ("2 of 4 test cases done"). If Claude stops part-way, what was reviewed is still saved and marked "Incomplete".

VERDICT	MEANING
Correct	Traceable to a requirement; steps, precondition, persona and expected result match it. Must name the requirement(s).
Needs update	Traceable and mostly right, but something should change (incomplete expected value, missing step, weak precondition, wrong persona). Must list the issues and give corrected text.
Incorrect	Contradicts the requirement, so the test would pass wrong behaviour or fail correct behaviour. Must list the issues and give corrected text.
Not traceable	No requirement in the document covers it.

Verified end to end. On a sample BRD with five requirements and four test cases, the review correctly marked TC_010 Incorrect (expects the Five Below Flag to default to True; the BRD says False), TC_014 Needs update (misses the "Prospect" default), TC_034 Correct and TC_050 Not traceable (nothing covers deleting Accounts), and proposed four missing test cases, in about 30 seconds.

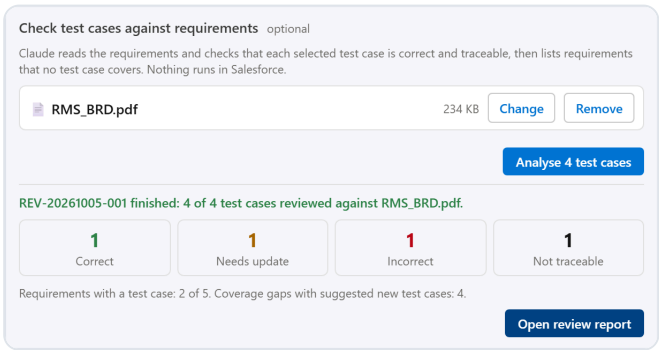


Figure 8. The requirements check in the panel after a review (illustrative data).

9 Reports

All results are kept in plain data files in the `qa/` folder, the source of truth. Three report pages read them and are opened from the panel through a secret link. Each report starts with a one-line verdict and a plain-English summary for management, then key figures, charts and the detail; each can be filtered, exported to CSV and printed to PDF.



Figure 9. Report data flow. Reviewers can change a run or bug status on the page (with a note and their name); the change is kept in the browser until they save it back into the data file.

REPORT	WHAT A MANAGER SEES FIRST	DETAIL AVAILABLE
Test Execution	Verdict (On track / At risk / Needs attention), pass rate, results bar, open defects to watch, results by module. Counts the latest result of each test case by default; "All runs" includes re-runs.	Per test: expected vs actual, steps, screenshots, status history, status change.
Defect Report	Open defects, Critical / High count, open defects by severity and status, defects by module, average age.	Steps to reproduce, screenshots, every run that hit the bug.
Test Case Review	Verdict (Ready to test / Minor / Significant rework), % correct, requirement coverage, missing test cases.	Issues per test case with current and suggested text side by side; earlier reviews.

The reports as they look today

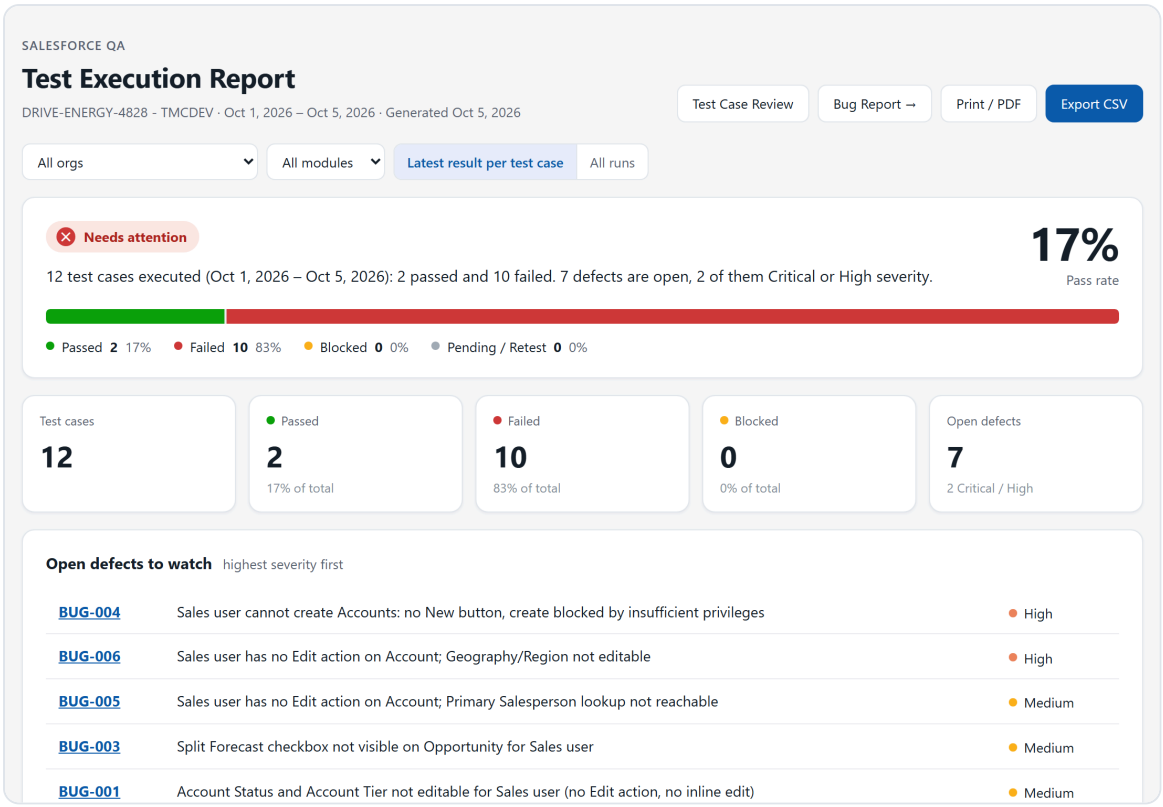


Figure 10. Test Execution Report with live data from the TMCDEV sandbox (12 test cases, 17% pass rate, 7 open defects).

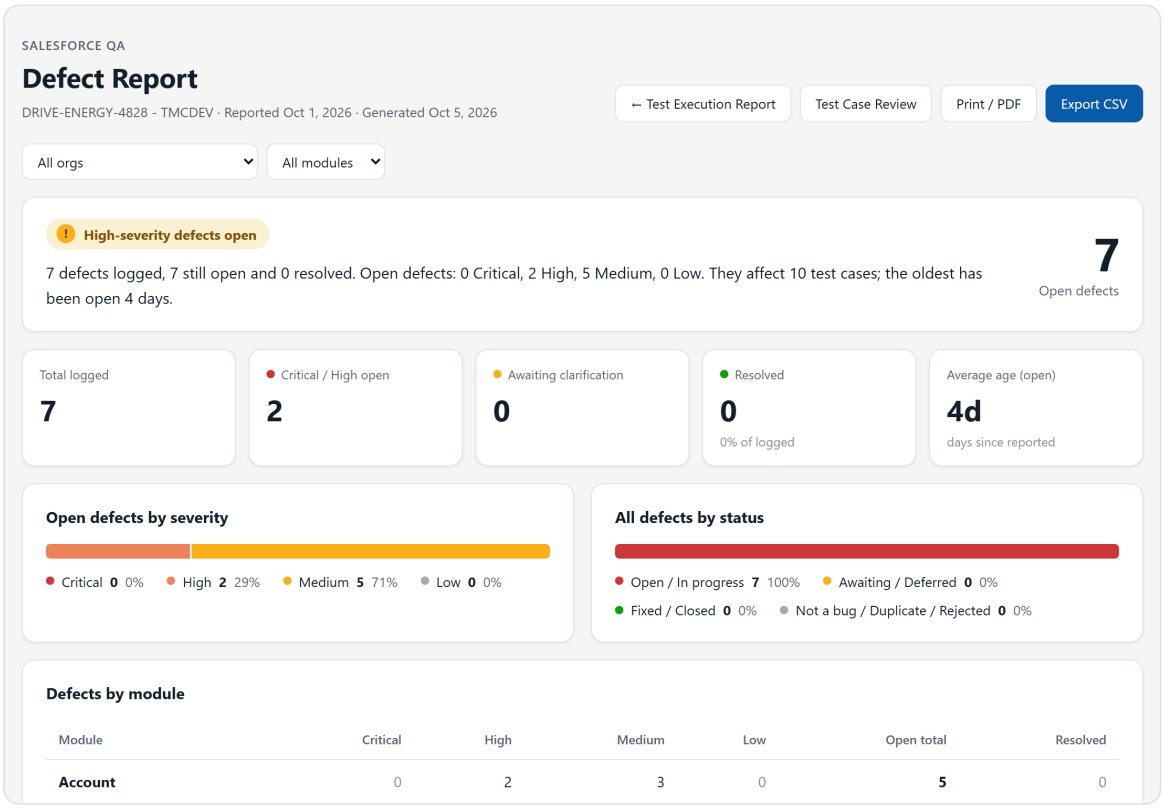


Figure 11. Defect Report with live data: 7 open defects, 2 High; most concern Sales-profile access on Account.

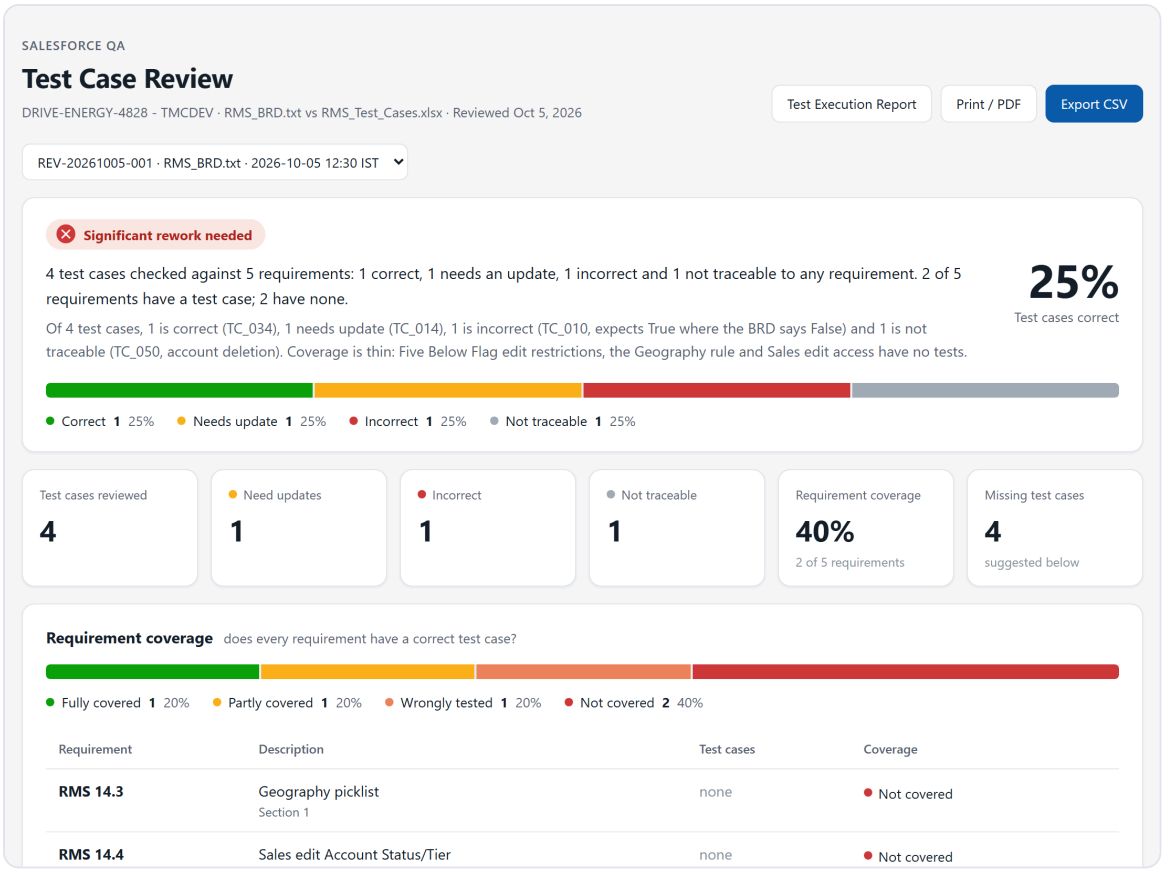


Figure 12. Test Case Review from the end-to-end check on a sample BRD: 1 of 4 test cases correct, 2 of 5 requirements with a test case, 4 missing test cases suggested.

10 Security and safeguards

AREA	SAFEGUARD
Network exposure	The server listens on 127.0.0.1 only and rejects requests from other hosts and from web pages' origins; it cannot be reached from the network.
Who can call the server	Every API call needs the agent token (compared in constant time). The token lives in the extension's background worker, never in the Salesforce page. Reports use a separate unguessable link key derived from the token.
Claude's tools	Claude Code runs with no built-in tools (no shell, no file reads or edits). Test runs get only the browser and QA tools; reviews get only the review tools.
Tool-call isolation	Each run or review gets its own random key, so a Claude process can only report into its own run. Every tool call is schema-validated.
Salesforce credentials	No persona passwords exist. The admin session is saved locally under `auth/`; the persona session file is deleted after every run, and any left over from an interrupted run is removed at start-up.
Your own browser	Tests run in a separate Playwright browser; the tester's Chrome session is never used or changed.
Test data	Claude may read with single, bounded SOQL SELECT queries; may create only business records (standard business objects and custom objects); may edit only records created for testing; may not delete anything or change Setup.

AREA	SAFEGUARD
Untrusted content	Test cases and BRDs are passed to Claude as data, with instructions to ignore any instructions inside them; panel text is rendered without HTML, so pasted text cannot inject markup.
AI usage and cost	Claude runs on the signed-in Claude Code seat (Sonnet by default); API keys are deliberately not passed to it. Runs have a turn limit and a 30-minute timeout.

11 Operating the agent

Daily use

1. Start the server in a terminal and keep it open: `npm start`
2. Open any Salesforce page and click the shield button (bottom right).
3. Check the org shows "connected"; click **Reconnect** if the admin session has expired.
4. Pick the persona, then paste a test case and click **Run test**, or upload a sheet in **Upload file**.
5. Optionally add the BRD and click **Analyse** before running the batch.
6. Open the reports from the panel when the run or review finishes.

Commands

COMMAND	PURPOSE
`npm install`	Install dependencies (first time).
`npm start`	Start the agent server.
`npm run build:extension`	Build the extension into `packages/extension/dist`; then reload it in `chrome://extensions`.
`npm run connect -- --org <id>`	Connect an org from the terminal.
`npm run connections -- --verify`	List orgs, admin connections and personas, and check each session still works.
`npm run check`	Typecheck, lint and the 127 automated tests.

Settings (.env)

SETTING	MEANING
`AGENT_TOKEN`	Shared secret pasted into the extension settings (at least 16 characters).
`PORT`	Server port, default 4317.
`BROWSER_CHANNEL`	`chrome` (installed Chrome) or `chromium`.
`HEADLESS`	`false` to watch test runs in a visible window.
`CLAUDE_MODEL`, `CLAUDE_MAX_TURNS`, `RUN_TIMEOUT_MINUTES`	Optional: model (default sonnet), turn limit (150) and per-run timeout (30 minutes).

Where things are kept

`config/orgs.json`	Detected orgs and their personas.
`auth/<org>/`	Saved admin session per org (sensitive: never share).
`qa/`	Reports, data files, screenshots, bug evidence, test-data registry.
`runs/`	Raw record of each run (run.json).

12 Limitations and next steps

Current limitations

- **Runs on one machine.** The server runs on the tester's PC and must stay running in a terminal; there is no shared or hosted server yet.
- **One test at a time.** Runs are sequential by design (shared admin session); a large batch takes as long as its tests combined.
- **One review at a time;** up to 200 test cases and about 400,000 characters of BRD text per review. Scanned (image-only) PDFs cannot be read; Word .doc and Excel .xls must be re-saved as .docx / .xlsx.
- **AI judgement.** Claude's step results and review verdicts are evidence-backed but should be spot-checked; reviewers can change any status in the reports with a note.
- **Reports are local files.** They are shared by link only while the server runs, or by printing to PDF / exporting CSV.
- **Data quality.** Module names come from the test sheets, so variants such as "Account" and "Accounts" appear separately until the sheets are made consistent.

Suggested next steps

1. Run the agent server as a background Windows service so it does not depend on an open terminal.
2. Use the requirements review on each new sprint's test sheet before execution, and add the suggested missing test cases.
3. Agree a triage routine for the 7 open defects, starting with the two High-severity Account access issues (BUG-004, BUG-006), and confirm with the business which are intended Sales-profile restrictions.
4. Standardise module names and persona labels in the test sheets.
5. Consider a shared, access-controlled location for the reports so management can open them without the tester's machine.

> **Note on the README.** The project README still describes the agent as "Phase 1 (walking skeleton)" and says the server holds a Claude API key. Both are out of date: AI execution is in place and Claude runs on the Claude Code seat. Updating that section is a small follow-up.